EF Core: Tracing and Logging Queries

Organized lesson summary | SQL Preview vs. Runtime Logging

1. Core Idea

When you write a LINQ query, Entity Framework Core translates it into SQL that can be executed against the database. Tracing helps you inspect the generated SQL, understand query behavior, and investigate performance issues instead of guessing.

```
LINQ Query → EF Core Translation → SQL → Database Execution → Results
```

2. Two Ways to Inspect Queries

ToQueryString()	Runtime Logging
Previews the SQL for the current IQueryable pipeline before the query executes.	Records EF Core commands during actual execution. Depending on configuration, logs can include SQL, parameters, and timing information.
Best for: • Understanding LINQ translation • Reviewing WHERE, JOIN, and ORDER BY • Inspecting query shape	**Best for:** • Seeing SQL executed at runtime • Inspecting Count, Any, First, etc. • Investigating execution and performance

3. Using ToQueryString()

It works well with queries that still return rows, including Where(), Select(), OrderBy(), Join(), Include(), Skip(), and Take().

```
var query = context.Students
    .Where(s => s.Status == "Active")
    .OrderBy(s => s.StudentId);

Console.WriteLine(query.ToQueryString()); // Preview SQL
var students = query.ToList();             // Execute query
```

Important: ToQueryString() is a preview of the query shape. It does not guarantee that the displayed SQL is always exactly the final command sent to SQL Server.

4. Why Runtime Logging Matters

Terminal or scalar operators can change the SQL needed at execution time. Examples include:

- Count() / LongCount() may produce a query using COUNT(*).
- Any() may produce an existence check, such as EXISTS.
- First() / FirstOrDefault() may limit results, such as with TOP(1) on SQL Server.
- Sum(), Average(), Min(), Max(), and Single() may change the SQL shape or result-handling behavior.

Use runtime logging to verify what actually ran, especially when inspecting these operations.

5. Enabling Logging in a Console App

```
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Logging;

var options = new DbContextOptionsBuilder<AppDbContext>()
    .UseSqlServer(connectionString)
    .LogTo(Console.WriteLine, LogLevel.Information)
    .Options;
```

- UseSqlServer(connectionString): configures the SQL Server provider and connection string.
- LogTo(Console.WriteLine, LogLevel.Information): writes EF Core log messages to the console at Information level.

- EnableSensitiveDataLogging(): can expose parameter values during development, but may reveal confidential data. Avoid enabling it in production unless there is a carefully controlled reason.

Remember to include using Microsoft.Extensions.Logging; when using LogLevel.Information.

6. Typical SQL Shapes in Logs

```
-- Query returning rows
SELECT [s].[StudentId], [s].[Status], ...
FROM [Students] AS [s]
WHERE [s].[Status] = N'Active'
ORDER BY [s].[StudentId];

-- Count()
SELECT COUNT(*)
FROM [Students] AS [s]
WHERE [s].[Status] = N'Active';

-- First() on SQL Server (typical shape)
SELECT TOP(1) ...
FROM [Students] AS [s]
...
```

Exact SQL, column names, and formatting depend on your model, EF Core version, and database provider.

7. Common Mistakes

- Assuming ToQueryString() always shows the exact final SQL executed.
- Skipping runtime logging and trying to diagnose behavior by guesswork.
- Forgetting that Count(), Any(), First(), and aggregate methods can produce different SQL.
- Leaving sensitive-data logging enabled in production without proper safeguards.

8. Key Takeaway

```
ToQueryString()  =  Blueprint (preview the SQL shape)
Logging          =  Runtime evidence (commands executed)
```

Use ToQueryString() to understand query design and LINQ translation. Use runtime logging to verify executed commands and investigate runtime behavior or performance.